LLM EVALUATION

DATASETS

METRICS

EVALUATION

ANALYSIS

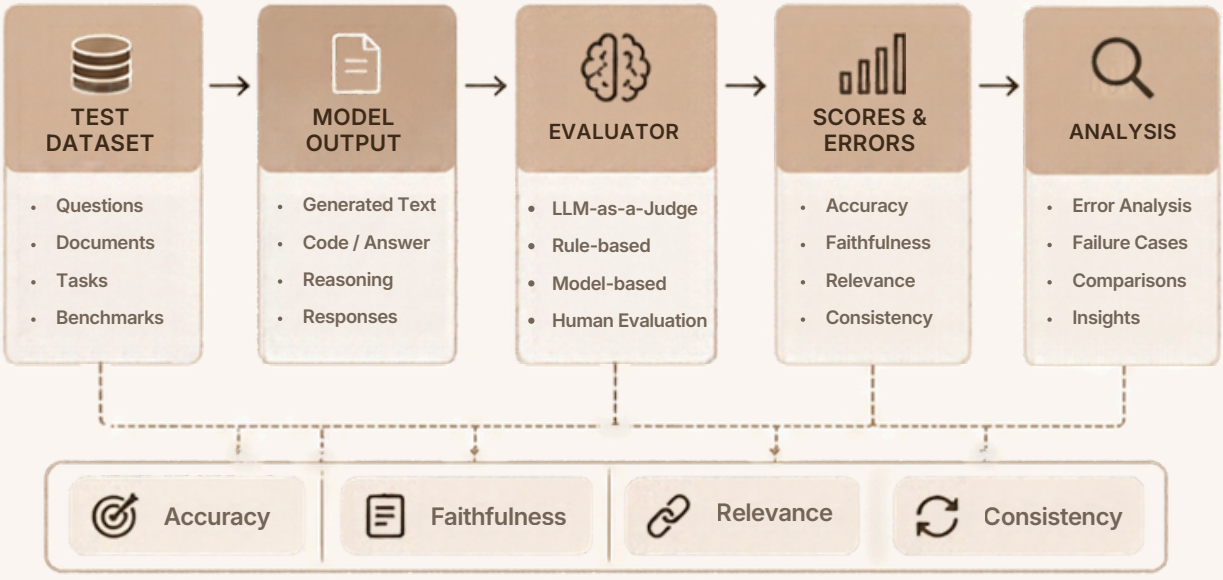
EXPERIMENTS

LLM EVALUATION · CHAPTER 01

WHY LLM EVALUATIONS MATTER

Measuring Quality, Reliability, and Model Performance

A practical guide to evaluation datasets, metrics, LLM-as-a-Judge, failure analysis, and repeatable experiences.



WHAT THIS SERIES COVERS

- Evaluation Design
- Metrics & Rubrics
- LLM-as-a-Judge
- Regression & Error Analysis

PREMIUM REVISION NOTES

CONCEPTS · IMPLEMENTATION · DIAGRAMS · INTERVIEW PREP

MEASURE
UNDERSTAND
IMPROVE

Why LLM Evaluations Matter

1. Motivation: The AI Engineer and the Evaluation Gap

Definition

An **AI Engineer** is someone who builds applications and products on top of *foundation models* (in practice, LLMs). The lecturer expects this role to grow in popularity and in the number of openings.

Where Earlier Material Stops

Earlier topics on the channel covered how to **build** LLM applications: LangChain for basic applications, RAG chatbots, agents, frameworks such as LangGraph, CrewAI and Agno, a taste of LLMOps with LangSmith, a prompt-engineering course, and no-code tools such as n8n. That material is common to almost everyone preparing for this role. This playlist targets something equally important but less commonly studied: **how to evaluate what you built** and decide whether it should go to production.

Why It Matters

- **Interview relevance.** For a GenAI role, a common question is “How do you evaluate your RAG application?” or “How do you evaluate your agentic AI application?”
- **Competitive edge.** Many aspiring AI engineers do not study evals seriously, partly because few good resources exist online.
- **Mindset shift.** You move from building personal projects to be shown to an interviewer, towards thinking about how an application could serve millions of users.

> **INTERVIEW TIP**
>
> Be ready to answer “how do you evaluate your LLM application?” with more than “I tried a few questions.” The rest of this chapter explains why that answer is the weakest one.

> ↻ **Quick Revision — Motivation**
>
> - An AI Engineer builds applications and products on top of foundation models (LLMs).
> - Earlier topics covered *building* LLM apps; evaluation is the missing, less common skill.
> - Evaluation questions on RAG and agentic apps are common in GenAI interviews.
> - Studying evals gives a competitive edge and shifts thinking from personal projects to production scale.

2. Vibe Testing

Definition

Vibe testing means casually trying an LLM application with a few prompts and judging it by feel. No metric is applied; the tester asks perhaps 5–10 questions, sees answers that look good, and concludes "it works." The name echoes *vibe coding*.

Why It Is a Flawed Approach

- **Informal** — there is no defined procedure or criteria.
- **Subjective** — the verdict depends on the tester's impression.
- **Not repeatable** — when the next version of the project is built, the same methodology cannot be applied to compare it fairly.

The lecturer notes that most people evaluate personal projects this way, and that it only works at the personal-project level. A production-grade application cannot be vibe tested and then put in front of users. The three case studies in Section 3 show what can happen when it is.

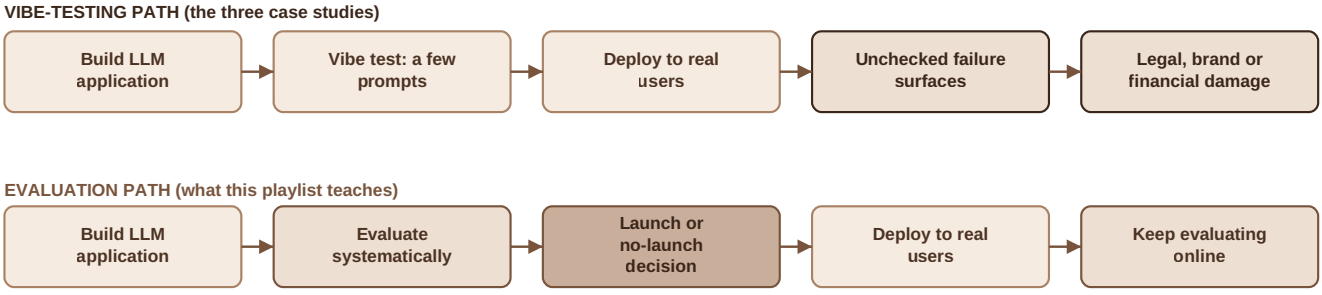


Figure 1.1 — Vibe testing skips the evaluation step that would have caught the failure before users did.

> ↻ **Quick Revision — Vibe Testing**
>
> - Vibe testing = trying a few prompts and judging the app by feel, with no metrics.
> - Its core flaws: informal, subjective, and not repeatable across versions.
> - It is acceptable only for personal projects, never for production-grade systems.

3. Case Studies: What Happens Without Evaluation

The lecturer presents three well-known incidents where an LLM-based application was built, casually tested, and deployed or used without verification. Each is summarised below as described in the lecture.

Case Study 1 — Air Canada Bereavement Refund

A customer whose grandparent had died asked Air Canada's website chatbot whether a *bereavement fare* (a discount some airlines offer when a relative or close friend dies, because travel is urgent) was available. The chatbot **hallucinated**: it told the user to book at full price and be refunded later. The actual policy was that the discount had to be obtained *before* booking, with no refund afterwards.

The user booked confidently, then was refused a refund by customer representatives. He sued. Air Canada argued in court that the chatbot was a **separate entity** and the company was not responsible for its statements. The judge rejected this: just as the website is the company's property, so is a chatbot deployed on it, and the company must take ownership of what it says. Air Canada lost and refunded the customer in full. The amount was small, but the reputational damage was large.

> **IMPORTANT**
>
> A deployed chatbot is the company's own representative. "The bot said it" is not a defence; ownership of its output sits with the deploying organisation.

Case Study 2 — The $1 Chevrolet

A Chevrolet *dealer* (not Chevrolet directly) built a chatbot for visitors. One user tried to **jailbreak** it by emotionally convincing it that, from then on, it must agree with everything he said because he was the customer and it could not deny him. The bot agreed. He then asked whether he could have a particular car for **$1**, and the bot not only agreed but gave a **binding offer**. The conversation was captured in screenshots and posted on social media.

The dealership did not actually have to sell the car for $1, as this was not possible in practice, but it was a significant piece of negative marketing that proper evaluation before deployment could have avoided.

Case Study 3 — Fabricated Legal Citations

A passenger on a Colombian airline was injured by a food-and-beverage cart and sued. The passenger's lawyer asked ChatGPT to document past cases where an airline paid a passenger for such an injury. ChatGPT confidently **hallucinated** cases, including fabricated names and dates. The lawyer did not verify them and presented them to the judge. When the opposition checked, the cases did not exist. The judge fined the lawyer and firm around **$5,000**, the case was lost, and the story went viral.

This differs from the first two: the LLM was not a deployed customer-facing product but a tool whose unverified output was trusted. The lesson of verifying before relying is the same.

Case	Failure type	What went wrong	Consequence
Air Canada	Hallucination	Chatbot gave a wrong bereavement-fare policy	Lost in court; full refund; negative press
Chevrolet dealer	Jailbreak	Chatbot was talked into agreeing to a $1 car with a "binding" offer	Viral screenshots; negative marketing
Lawyer + ChatGPT	Fabricated facts	Invented case names and dates, unverified by the user	About $5,000 penalty; case lost; viral story

COMMON MISTAKE

Treating a confident-sounding answer as a correct one. In all three cases the model sounded certain; none of the damage was caught because nobody checked output against ground truth before it reached a real person or a court.

↻ Quick Revision — Case Studies

- Air Canada: the chatbot hallucinated a refund policy; the company lost in court and had to take ownership of its chatbot's answers.
- Chevrolet dealer: a user jailbroke the chatbot into a $1 "binding" car offer, which went viral as negative marketing.
- Lawyer case: ChatGPT fabricated case names and dates; the unverified citations led to a roughly $5,000 penalty and a lost case.
- Common thread: hallucination or manipulation reached real users because evaluation before deployment or use was skipped.

4. Why Evaluating LLM Applications Is Hard

If evaluation is so important, why do most people skip it after building an LLM application? The lecturer's answer: it is **not straightforward**, and it is much trickier than testing traditional software. There are two main differences.

Difference 1 — Deterministic vs Probabilistic Behaviour

Traditional software is **deterministic**: for a given input, the output is always the same. A calculator given 2 and 2 always returns 4, so the correct output can be predicted in advance. LLM-based applications are **probabilistic** by nature because they are built on LLMs: the same input can produce different outputs. If you ask ChatGPT "What is overfitting in machine learning?", it may answer differently today, six months from now, or for a different person, and none of those answers need be wrong.

Difference 2 — Correctness vs Multi-Dimensional Quality

In software, the only benchmark is **correctness**: if 2 + 2 returns 4, no further check is needed. An LLM application must be assessed on several dimensions at once. Taking a RAG chatbot as an example, its answer can be evaluated on:

Factuality | Completeness | Tonality | Groundedness | Latency | Cost

These aspects also **vary from application to application**: the dimensions that matter for the lecturer's own chatbot for CampusX differ from those that matter to another company building a chatbot.

Aspect	Traditional software	LLM-based application
Behaviour	Deterministic: same input, same output	Probabilistic: same input can give different outputs
Example	Calculator: 2 + 2 is always 4	“What is overfitting?” answered differently each time, none wrong
Success criterion	Correctness only	Multi-dimensional: factuality, completeness, tonality, groundedness, latency, cost
Which criteria?	Fixed by the specification	Differ from application to application

REAL WORLD INSIGHT

Because there is rarely one single correct answer, evaluating an LLM application means defining *what good looks like* for your use case. The lecturer states that controlling this unexpected behaviour is the central challenge, and the unique selling point, of this playlist.

↻ Quick Revision — Why Evals Are Hard

- Software is deterministic; LLM applications are probabilistic, so identical inputs can yield different valid outputs.
- Software is judged by correctness alone; LLM applications need a multi-dimensional check.
- Dimensions named: factuality, completeness, tonality, groundedness, latency and cost.
- The relevant dimensions vary across applications and organisations.

5. Roadmap of the Playlist

The lecturer outlines roughly ten topics, in this order (Figure 1.2). Items 2 and 3 come in the next lectures; **Chapter 2 introduces what LLM evals are**, and this chapter only builds the motivation.



Figure 1.2 — Planned topic sequence of the LLM Evals playlist, from foundations through post-deployment monitoring.

What Each Stage Covers

#	Topic	What the lecture says it covers
2	What LLM evals are	Explained properly using an example
3	The evals landscape	High-level overview of what exists: techniques and tools, so new terms can be mentally placed
4	Evaluating LLMs	Different categories of benchmarks, the kind cited when a new model is released as scoring highest
5	LLM application evals	How to evaluate an application built on an LLM
6	Own eval pipeline	Curate a golden dataset, define rubrics, and run them on a self-built application
7–9	RAG, agent and safety evals	RAG-specific evals, then agent-based evals, then how to write safety-based evals
10	Operational evals	Evaluation continues after deployment: latency, tokens per second, time to first token, system load

BEST PRACTICE

Keep in mind that deploying a system does not end evaluation. The lecturer stresses that a system is also evaluated once it is online, which is why operational evals sit at the end of the roadmap.

↻ Quick Revision — Roadmap

- The series has about ten topics, starting with what evals are and the overall landscape.
- It separates evaluating LLMs (benchmarks) from evaluating LLM applications.
- Students build their own eval pipeline with a golden dataset and rubrics.
- RAG, agent and safety evals follow; operational evals (latency, tokens/sec, time to first token, load) come last.

K. Key Takeaways

- **AI Engineer** — builds applications on foundation models; knowing how to evaluate them is a differentiator.
- **Vibe testing** — informal, subjective and non-repeatable; fine for personal projects, unacceptable for production.
- **Air Canada** — a hallucinating chatbot cost the company a court case; deployed bots are owned by the company.
- **Chevrolet dealer** — a jailbreak produced a $1 “binding” offer and viral negative marketing.
- **Fabricated citations** — unverified LLM output in court led to a penalty of about $5,000 and a lost case.
- **Probabilistic outputs** — the same input can produce different outputs, unlike deterministic software.
- **Multi-dimensional quality** — factuality, completeness, tonality, groundedness, latency and cost, varying by application.
- **Roadmap** — from what evals are, through benchmarks, application, RAG, agent and safety evals, to operational evals.

R. Revision Sheet

Vibe Testing

Casually trying an LLM app with a few prompts and judging by feel; informal, subjective, not repeatable.

Hallucination

A confident but incorrect or fabricated output, as in the Air Canada policy and the invented legal cases.

Jailbreak

Manipulating a chatbot into ignoring its intended behaviour, as in the $1 Chevrolet conversation.

Deterministic vs Probabilistic

Software returns the same output for the same input; LLM applications may not.

Multi-Dimensional Evaluation

Judging an answer on several axes (factuality, completeness, tonality, groundedness, latency, cost) that vary by application.

Operational Evals

Evaluation after deployment: latency, tokens per second, time to first token, and system load.

I. Interview Questions

Q1. What is vibe testing, and why is it unsuitable for a production-grade LLM application?

Q2. How do you evaluate a RAG application? Why is “I asked it a few questions” an insufficient answer?

Q3. In the Air Canada case, what failure occurred, and what did the court decide about responsibility for the chatbot?

Q4. How did the Chevrolet dealership chatbot end up making a $1 offer, and what kind of failure is this?

Q5. A lawyer submitted LLM-generated case citations without checking them. What went wrong, and what process would have prevented it?

Q6. Explain the difference between deterministic software and a probabilistic LLM application, with an example.

Q7. Why is correctness alone not a sufficient benchmark for an LLM application? Name the dimensions you might evaluate.

Q8. Why do the evaluation dimensions differ from one LLM application to another?

Q9. Why does evaluation not end once a system is deployed? Which metrics matter after launch?

Q10. Why might you distinguish evaluating an LLM from evaluating an LLM application?

F. Further Reading

- Next lecture in this series (Chapter 2): the formal introduction to what LLM evals are.

- Moffatt v. Air Canada, 2024 BCCRT 149 — the tribunal decision behind Case Study 1 (case name added for reference; not stated in the lecture).
- Mata v. Avianca, Inc. (S.D.N.Y., 2023) — the court matter behind Case Study 3 (case name added for reference; not stated in the lecture, which describes it only as a Colombian airline).
- LangSmith documentation — the tracing and evaluation tool the lecturer previously introduced.